

CSE638: Graduate Systems
Midsem Winter 2026 (SOLUTIONS)
Date: February 24, 2026; Duration: 60 mins (10 am to 11 am)
Total: 29 points

Instructions:

- (1) Read the questions carefully and answer.
- (2) **Answer to the point as much as possible.** Include the important technical details.
- (3) If you make practical & appropriate assumptions, mention them clearly.

Q.1. [CO1] Answer the following: **[1+2+2+1 = 6]**

- (a) What type of address is generated by the CPU?
- (b) How can a user program written in a high-level language invoke the kernel code?
- (c) How does the user program obtain the address of kernel code? Is this address physical or virtual?
- (d) Comment on the address of some kernel function "koo" as seen by two different processes, A and B.

Ans.

- (a) The CPU generates a virtual address (also called a logical address).

Explanation (not required): When a user program executes an instruction that references memory, the CPU computes a virtual address. This virtual address is translated by the MMU (Memory Management Unit) using page tables. The MMU produces the physical address used to access DRAM.

- (b) Through a system call (trap) that switches the CPU from user mode to kernel mode, or call a library routine that in turn invokes a system call.

- (c) [Answer from one of the two cases is acceptable]

Case A — Normal system calls (common)

User program does not know kernel function addresses.

Instead, it executes a syscall; Hardware jumps to a fixed kernel entry; Kernel dispatches internally

So: The user does NOT need the kernel function address. The address is virtual.

Case B — When kernel is mapped (typical modern OS)

In many OSes (e.g., Linux), the kernel is mapped into every process's virtual address space, but marked supervisor-only

Thus, the address is virtual; the user mode cannot access it (page protection fault).

- (d) In Linux and most UNIX systems, the kernel is mapped at the same virtual address in every process, but it is protected by privilege bits. Therefore, the virtual address of the kernel function "koo" is the same for both (or all) the processes.

Q.2. [CO3] Explain why running a VM with QEMU+KVM is dramatically faster than running QEMU in full emulation mode. Identify which components run where. **[2+2 = 4]**

Ans.

Pure QEMU (full emulation)

- Guest instructions are binary translated
- Runs entirely in user space; No hardware virtualization
- Heavy instruction overhead

QEMU + KVM

Split design:

- KVM (kernel): runs guest code directly on CPU (hardware virtualization: VMX-mode)
- QEMU (user space): device emulation, VM management

KVM eliminates instruction translation and lets the CPU execute guest code natively.

Q.3. [CO3] Assume **4-level page tables** with the following details. **[2+1+2 = 5]**

Memory access time = 100 ns;

TLB miss rate = 1%;

Guest memory reference rate = 1 billion references/sec.

Compute the effective memory overhead **in terms of memory access time per sec** for:

A) Shadow paging

B) EPT (Extended Page Tables)

Assume:

Ignore TLB hits cost

Assume no page walk caching

Ans.

Step 1: Cost per page walk

Shadow paging

Single walk: **(4 memory accesses for page table) + (1 memory access for actual physical address) × 100 ns = 500 ns**

(Some may write 4 accesses, since they ignore the actual physical memory access. Give points if the explanation holds.)

0.5 point: #memory accesses

0.5 point: right answer

EPT

Two-dimensional walk:

guest walk: 4 accesses

EPT walk per level: 5 accesses

Total accesses = $5 \times 5 = 25$

Explanation:

Each guest PTE fetch itself requires an EPT walk.

So: $25 \times 100 \text{ ns} = 2500 \text{ ns}$

(Allow 4×4 , i.e., $16 \times 100 \text{ ns} = 1600 \text{ ns}$)

0.5 point: #memory accesses

0.5 point: right answer

Step 2: Misses per second 1 point

TLB miss rate = 1%

Memory references/sec = 10^9

So misses/sec: $0.01 \times 10^9 = 10^7$ misses/sec

Step 3: Total overhead per second

Shadow paging

$$10^7 \times 500 \text{ ns}$$

$$= 5 \times 10^9 \text{ ns}$$

= 5 seconds of stall per second 1 point

(Allow 4 seconds)

EPT

$$10^7 \times 2500 \text{ ns}$$

$$= 2.5 \times 10^{10} \text{ ns}$$

= 25 seconds of stall per second 1 point

(Allow 16 seconds)

Q.4. [CO1] List the sequence of steps that a packet goes through once it is received on the destination NIC's buffer until it reaches the destination process. **[4]**

1 point for each step; ignore the last step

Ans.

1. The DMA copies the packet from the NIC buffer into the NIC's kernel (packet buffer), and the RX ring is populated with the packet metadata + pointer
2. NIC triggers an interrupt (to attend to the packet)
3. Host CPU switches from user to kernel mode, performs top-half processing (i.e., acknowledges the interrupt), schedules the bottom-half processing for later, and returns to the user mode
4. When the bottom-half is scheduled:
 - a. sk_buff is dynamically created to hold the packet metadata
 - b. The Ethernet header is removed
 - c. Layer 3 and Layer 4 processing is done.
 - d. Layer 4 processing involves: handling the TCP state machine, enqueueing the skb to the socket's read queue, and sending a signal to the socket
 - e. On the signal, the application switches mode, dequeues the skb from the read queue, copies the packet from the packet buffers to the application buffers in the user space, releases the sk_buff, and returns to user mode.
5. The application can now use the received packet.

Q.5. [CO1] [2+1+2 = 5]

- (a) If you are asked to create a thread (similar to the pthread library), where the thread shares the memory space with the parent, will you use fork() or clone() system call? Why?
- (b) Suppose you are asked to perform malware analysis on your machine. You need to run a program that must not access the host network. Which system call would you use to detach your running process from the host network?
- (c) Suppose Process "A" creates a child process "B" and places it in a new namespace. Process "B" creates a child process "C".
- (c. i) What is the PID of process "B" in the new namespace?
- (c. ii) If process "B" exits unexpectedly, who reaps process "C"?

Ans.

(a) clone() → parent and child will be placed in different namespaces (isolation). We can specify shared components, such as memory and file descriptors, in the system call.

(b) unshare() → caller moves to new namespace

(c. i) 1

(c. ii) The kernel, i.e., the host process with PID=1, i.e., the init process at the host. The kernel sends SIGKILL to all remaining processes in the namespace and reaps them.

Q.6. [CO1]

(a) Why do we need both MAC and IP addresses for Internet communication? Why is just the MAC address or just the IP address not enough? **[1*2 = 2]**

(b) Why do we need port addresses? **[1]**

(c) Can you think of any other types or subtypes of the addresses discussed above needed for communication, say, from the IIITD network to the Google web server? If yes, what is the purpose of such an address? **[2]**

Ans.

(a) Only MAC addresses?

Flat address space → routing tables with billions of entries.

No aggregation → Internet routing breaks.

Works only for small, local networks.

Only IP addresses?

Hardware forwarding is simpler and faster with MAC addresses in a multihop local network. IP forwarding is complex, i.e., IP address to network address translation, and selects the path with the longest prefix, making the hardware expensive as well.

(b) Port address uniquely identifies an application/process once the IP packet reaches the machine. Used to deliver the message to the intended process.

(c) Internal networks typically use private IP addresses for isolation, etc. However, to communicate with the outside, the source IP address must be a public IP address. NAT routers/gateways are used to map the private and public IP addresses during communication.

THE END